

BlockPack: Block-Local Probe Packing for SSD-Resident Engram Memory

[Author Name]

Abstract

Engram-style conditional memory is attractive for large language models because its retrieval indices are deterministic, enabling precomputation, host-side prefetch, and memory-tier decoupling. However, the original multi-head hashed cold path is naturally designed for RAM-like random access rather than SSD-friendly block access. A naive SSD implementation can therefore turn one logical lookup into multiple scattered reads. This paper introduces *BlockPack*, a physical-layout companion for SSD-resident Engram memory. BlockPack does not change Engram’s lookup semantics, routing policy, or branch topology. Instead, it constrains a bounded candidate neighborhood to one aligned storage extent so that the cold path resolves through a block-local transaction rather than a probe-count-proportional series of reads. The proposal is compatible with the original Engram design, with Engram-Nine’s collision-free hot tier, and with multi-branch backbones such as mHC, because all three preserve deterministic key formation. The main claim is therefore architectural rather than algorithmic: if conditional memory is to scale into NVMe tiers, its physical layout must preserve bounded block-level service behavior. We formalize the method, clarify its compatibility boundaries, and provide a benchmark protocol for measuring mean, tail, and overlap-aware latency under realistic SSD constraints.

1 Introduction

DeepSeek’s Engram frames conditional memory as a complementary sparsity axis to conditional computation: local N -gram patterns are retrieved through deterministic lookup rather than reconstructed through repeated Transformer computation. This design is attractive not only for modeling reasons, but also for systems reasons. Because lookup IDs depend only on the token sequence, they can be computed before the corresponding layer executes, prefetched from host memory, and overlapped with preceding compute. The original paper further motivates a multi-level cache hierarchy in which frequent items live in faster tiers and the long tail can be placed in slower, higher-capacity media such as NVMe SSD. Those properties make Engram unusually amenable to storage hierarchy co-design.

The challenge is that *deterministic addressing alone does not guarantee SSD efficiency*. An SSD favors a small number of aligned, block-sized reads. By contrast, a naive realization of multi-head hashed lookup can splinter a single logical access into several small, far-apart reads. This mismatch is most acute in the cold tier, where cache locality is weakest and random-access penalties matter most.

A closely related follow-up, Engram-Nine, replaces the high-frequency collision-prone region with a collision-free hot tier built on a minimal perfect hash function (MPHF), while retaining the original hashed cold tier. Importantly, that work reports that improving hot-tier index precision does not consistently improve validation loss under its controlled setup, and argues that collisions may sometimes act as implicit regularization. This result is highly relevant for systems positioning: it suggests that the next systems bottleneck may lie less in hot-tier precision and more in how the

cold path is physically served. BlockPack is intended precisely as a *cold-tier serving primitive*, not as a replacement for Engram-Nine’s hot tier.

Finally, Engram is evaluated on a multi-branch backbone integrated with Manifold-Constrained Hyper-Connections (mHC), but the module is explicitly described as topology-agnostic. This distinction matters. mHC affects residual-stream stability and the compute available for overlap; it does not alter deterministic key formation. Consequently, the storage-layout problem studied here is orthogonal to whether the surrounding backbone uses vanilla residual streams, mHC, or a later alternative such as sHC.

Core thesis. BlockPack should be understood as a *physical-layout extension for the SSD-resident cold path of Engram-style memory*. It is compatible with:

- **Engram**, because deterministic N -gram keys and host-side prefetch are preserved;
- **Engram-Nine**, because the hot tier can remain collision-free while the cold tier adopts BlockPack; and
- **mHC or related multi-branch backbones**, because branch topology affects fusion and overlap windows, not the key-to-address mapping itself.

2 Positioning Relative to Engram, Engram-Nine, and mHC

2.1 Engram

Engram retrieves static N -gram memory by tokenizer compression followed by deterministic multi-head hashing. The retrieved memory is then fused into the hidden state through context-aware gating and a lightweight convolution, and the full system is designed to support host-memory offload and overlap-aware prefetch. Thus, the original design already contains the systems premise on which this paper builds: deterministic IDs make storage hierarchy optimization possible.

2.2 Engram-Nine

Engram-Nine introduces a collision-free hot tier for the most frequent N -grams using an MPHF while keeping the long-tail cold keys on the original hashed path. Its most important lesson for the present paper is not that collisions are “solved,” but that hot-tier precision is not obviously the dominant training bottleneck. For serving, the consequence is straightforward: the hot tier can stay in HBM or DRAM, while the remaining cold tier still needs a block-friendly physical layout. BlockPack is proposed as that companion layout.

2.3 mHC and related backbones

The published Engram system uses a multi-branch backbone with branch-specific gating and, unless otherwise stated, integrates with mHC. This affects how much compute is available to overlap retrieval and may influence the best layer placement for an Engram module. It does *not* change the fact that lookup IDs are deterministic functions of the token stream. Therefore, mHC does not require a different BlockPack design. In end-to-end benchmarks, it only affects the overlap term in the latency model introduced below.

3 Problem Formulation

Consider an Engram-style cold lookup that, for a query key g_t , must evaluate a bounded set of candidate locations

$$\mathcal{C}(g_t) = \{c_1, \dots, c_K\}.$$

In a RAM-oriented implementation, the K candidates may be stored at unrelated physical offsets. If the memory tier is SSD, the service cost is then approximately proportional to the number of physically distinct reads required to resolve these candidates. The practical problem is therefore not merely the logical probe count K , but the number of physical transactions induced by those probes.

Let B denote the aligned storage extent used for one SSD transaction (for example, a 4 KB page or a larger 16 KB/32 KB read unit), and let s denote the byte footprint of one cold-tier record. For BlockPack to resolve a bounded candidate set in one extent read, the packing constraint is

$$K \cdot s \leq B.$$

This formulation is intentionally more general than a fixed “nine candidates in 4 KB” slogan. Whether a single extent can store all candidate records depends on the actual record representation: compact cold codes, compressed payloads, or pointer-rich metadata all imply different values of s .

4 BlockPack Layout

4.1 Design principle

BlockPack maps all candidates in a bounded neighborhood to one aligned extent. The extent stores the per-candidate metadata required to test or rank those candidates, and, when the byte budget permits, also stores the compact value payloads needed for direct cold-tier retrieval.

Formally, define a packing function

$$\psi : g_t \mapsto (b_t, \pi_t),$$

where b_t is an extent identifier and π_t is an intra-extent permutation describing where the candidate records are placed inside that extent. Instead of issuing K unrelated reads for $\mathcal{C}(g_t)$, the runtime issues one aligned read for extent b_t and resolves the candidate set locally.

4.2 Cold-tier retrieval path

A practical BlockPack-compatible cold path consists of the following stages:

1. **Key formation.** Build the canonical suffix N -gram exactly as in Engram.
2. **Route selection.** If a hot-tier membership structure reports a hot hit, serve the request from the collision-free hot tier (Engram-Nine style) or from the original fast tier.
3. **Extent selection.** Otherwise compute the cold-tier extent ID b_t and intra-extent candidate ordering π_t .
4. **Single-extent read.** Issue one aligned SSD read for extent b_t .
5. **In-extent resolution.** Test fingerprints or metadata for the K packed candidates, recover the winning entry, and materialize the compact value vector.

4.3 What BlockPack does *not* change

BlockPack does not alter:

- the semantic role of Engram as conditional memory,
- the use of context-aware gating,
- the use of a collision-free hot tier if Engram-Nine is present,
- or the surrounding residual-stream topology, including mHC.

It is therefore best viewed as a *storage-layout primitive*, not a modeling primitive.

5 Compatibility with Engram-Nine

The most natural deployment is a tiered design:

- **Hot tier:** collision-free MPHF tier in HBM or host DRAM for the most frequent keys;
- **Warm tier:** optional DRAM cache for recently used cold extents;
- **Cold tier:** BlockPack-organized SSD extents for the long tail.

This hierarchy aligns with both papers’ assumptions. Engram-Nine keeps hot membership static and preserves deterministic indexing. Engram already motivates a cache hierarchy shaped by Zipfian access skew. BlockPack simply makes the bottom tier obey the same block-locality discipline that the upper tiers approximate through caching.

One subtlety from Engram-Nine matters here: its analysis suggests that eliminating collisions may not improve training behavior and may even induce earlier hot/cold gating mismatches in deeper layers. BlockPack is unaffected by this modeling issue because it addresses the *serving path* of cold memory, not the training dynamics of hot-tier precision. However, the paper should explicitly state that a BlockPack result says nothing by itself about whether MPHF hot routing helps model quality.

6 Compatibility with mHC and Multi-Branch Fusion

Because Engram is stated to be topology-agnostic, BlockPack requires no special architectural change for mHC. The only end-to-end systems interaction is through overlap. Let

$$T_{\text{io}}$$

be the cold lookup service time of one request, and let

$$T_{\text{overlap}}(\ell, \mathcal{B})$$

be the amount of retrieval latency hidden by the compute performed before the Engram layer at position ℓ under backbone $\mathcal{B}$. Then the user-visible stall is

$$T_{\text{stall}} = \max\{0, T_{\text{io}} - T_{\text{overlap}}(\ell, \mathcal{B})\}.$$

A multi-branch backbone such as mHC may change T_{overlap} by changing compute intensity and layer placement, but it does not alter the address-generation logic of BlockPack. This is why BlockPack should be evaluated both as a raw I/O microbenchmark and as an overlap-aware end-to-end benchmark.

7 Evaluation Protocol

7.1 Variants

A clean benchmark should compare at least the following four storage variants:

1. **Hashed-SSD baseline:** original Engram-style multi-head cold lookup stored naively on SSD.
2. **Engram-Nine hot tier:** MPHf hot tier plus naive hashed cold path.
3. **BlockPack cold tier:** original hot path plus BlockPack-organized cold path.
4. **Engram-Nine + BlockPack:** collision-free hot tier plus BlockPack cold tier.

Optional ablations include Bloom-filter miss suppression, batched prefetch, layer placement, and overlap budget.

7.2 Metrics

The original draft reported relative lookup costs only. A more academic evaluation should add:

- mean latency per lookup,
- p50, p95, and p99 latency,
- number of physical read calls per logical lookup,
- bytes transferred per lookup,
- hot-tier hit rate and cold-tier miss rate,
- effective stall after overlap,
- throughput under prefill batching,
- and sensitivity to record width s , extent size B , and query skew.

7.3 Illustrative synthetic result

If the existing table from the earlier draft is retained, it should be labeled clearly as a *synthetic or simulator-based* result rather than a hardware result. For example, one may report that naive multi-probe lookup incurs a higher relative cost than a block-packed layout, while Bloom filtering and batching further reduce average service cost. That is a valid architectural pilot result, but it should not be presented as proof that a production Engram stack runs from SSD at near-RAM speed.

8 Main Technical Corrections to the Earlier Draft

The revised paper corrects four issues that would otherwise weaken the contribution.

Correction 1: rename the method. The earlier name “Sector-Nine” risks confusion with Engram-Nine while the two ideas solve different problems. The revised name “BlockPack” makes the role of the method explicit: it is a block-local physical layout for the cold tier.

Correction 2: generalize beyond “nine candidates in 4 KB.” A fixed claim of packing nine candidates into one 4 KB block is underspecified unless the record footprint is stated. The revised paper replaces that slogan with the packing inequality $Ks \leq B$, which makes the feasibility condition explicit.

Correction 3: narrow the claim from modeling to systems. The revised paper avoids implying that the method improves Engram’s training quality. The contribution is instead framed as a serving-path primitive for SSD-resident cold memory.

Correction 4: separate hot-tier precision from cold-tier locality. Engram-Nine is about collision-free hot indexing; BlockPack is about cold-tier physical locality. The revised paper treats them as complementary rather than as successive versions of the same idea.

9 Limitations

This work remains a systems proposal with a limited evidence base.

- Without hardware benchmarks on a real NVMe device, the current evidence supports architectural plausibility rather than definitive deployment claims.
- The benefit is expected to appear most strongly in tail latency and bounded read count, not necessarily in dramatic mean-latency reductions.
- If cold records are too large to satisfy $Ks \leq B$, a second-stage payload fetch or a larger aligned extent may be required.
- The method does not address Engram-Nine’s training-dynamics questions about gating mismatch, hot/cold flipping, or collision-induced regularization.

10 Future Directions

The revised framing suggests several concrete follow-ups.

1. **Overlap-aware end-to-end measurement.** Evaluate BlockPack under different Engram insertion depths and different backbones to quantify T_{overlap} .
2. **Compressed cold records.** Explore 8-bit, 4-bit, or product-quantized cold payloads to expand the feasible region of $Ks \leq B$.
3. **Extent-level caching.** Compare row caching versus extent caching under Zipfian access.
4. **Gating-aware prefetch.** Use gate estimates to prefetch only likely-to-be-used cold extents.
5. **Learned cold packing.** Instead of hash-only co-location, cluster cold keys with correlated access into the same extent.

11 Conclusion

BlockPack is best understood as a missing systems layer under Engram-style conditional memory. Engram provides deterministic retrieval, Engram-Nine optionally refines the hot tier, and mHC or other multi-branch backbones govern how retrieved memory is fused into the network. None of these, by themselves, guarantee that the SSD-resident cold path will remain block-friendly. BlockPack addresses that specific gap by reorganizing the long-tail memory layout so that one logical cold lookup can be served by one aligned extent read whenever the packing condition permits. This is a narrower claim than improving model quality, but it is also a cleaner and more defensible one.

References

- [1] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, *et al.* Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372, 2026.
- [2] Tao Lin. A Collision-Free Hot-Tier Extension for Engram-Style Conditional Memory: A Controlled Study of Training Dynamics. arXiv:2601.16531, 2026.
- [3] Zhaoyi Liu, Haichuan Zhang, and Ang Li. Beyond the Birkhoff Polytope: Spectral-Sphere-Constrained Hyper-Connections. arXiv:2603.20896, 2026.
- [4] Burton Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 1970.
- [5] Mendel Rosenblum and John Ousterhout. The Design and Implementation of a Log-Structured File System. *ACM Transactions on Computer Systems*, 1992.